

Hackathon Web App Setup Guide

React TypeScript Tailwind CSS v4 and Java 21 Spring Boot

Build a browser app and a Java REST API in one repository. This guide takes a new team from tool installation to a working request, then covers builds and deployment. Follow the sections in order; the example requires no database, authentication service, or paid hosting.

1 Install the development tools

Install VS Code, Git, Node.js 24 LTS with npm, and a Java 21 JDK such as Eclipse Temurin. Use the official installers linked in section 9. Enable the JDK installer options for JAVA_HOME and PATH where offered. Restart VS Code after installation so its terminal sees the new environment.

Node.js runs the frontend tools and development server. React runs in the browser; Spring Boot runs the API on the JVM. A production static frontend does not require a Node.js server.

In a new VS Code terminal, verify the tools:

Terminal

```
node --version
npm --version
java --version
javac --version
git --version
```

Expect Node 24.x and Java and javac 21.x. Vite currently requires Node 20.19+ or 22.12+; Node 24 LTS is the baseline for this guide. JAVA_HOME must point to the JDK directory, not its bin folder. Do not install Gradle globally; use the backend's generated wrapper. [1–4]

Set up VS Code

Open the repository root with File > Open Folder. Use Terminal > New Terminal for separate frontend and backend sessions. On Windows, the commands below use PowerShell. If npm.ps1 is blocked, use npm.cmd instead of npm rather than changing machine security settings. On macOS or Linux, use your normal shell.

Install these extensions from the Extensions view:

- Extension Pack for Java by Microsoft — Java editing, debugging and tests.
- Spring Boot Extension Pack by VMware — Initializr and Spring Boot support.
- Gradle for Java by Microsoft — Gradle tasks and project navigation.
- ESLint by Microsoft — frontend lint feedback.
- Prettier Code formatter by Prettier — consistent frontend formatting.
- Tailwind CSS IntelliSense by Tailwind Labs — utility completion and previews.

JavaScript and TypeScript support is built into VS Code. If Java import fails, run Java: Configure Java Runtime from the Command Palette and select JDK 21 for this project. Some Java extension versions need a newer JDK for the language server; keep the project's Gradle toolchain set to 21. [5]

2 Scaffold and configure the frontend

Start in the directory that will contain your repository. Run one command per line:

Terminal

```
mkdir hackathon-app
cd hackathon-app
git init
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm install -D tailwindcss@4 @tailwindcss/vite@4
```

If the generator asks to install packages and start the server, decline that optional step and run the explicit commands above. Keep the generated TypeScript and ESLint configuration. Commit package-lock.json so teammates can use npm ci. The @4 suffix keeps Tailwind on major version 4. [3, 6]

Replace frontend/vite.config.ts with:

TypeScript

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    port: 5173,
    strictPort: true,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
      },
    },
  },
})
```

Replace frontend/src/index.css with:

CSS

```
@import "tailwindcss";
```

Keep import './index.css' in frontend/src/main.tsx. Replace App.tsx with the example in section 5; it deliberately does not import the template's App.css.

Tailwind v4 uses the Vite plugin and the CSS import above. No tailwind.config.js, PostCSS configuration, or npx tailwindcss init -p is required for this setup. Use complete utility class names, such as bg-blue-600, in source files so Tailwind can detect them. [6]

The proxy preserves /api: a browser request to localhost:5173/api/hello reaches localhost:8080/api/hello. No rewrite is needed. This Vite server configuration applies during development; production routing is configured separately. [7]

3 Scaffold the Java backend

Open <https://start.spring.io> and select:

- Project: Gradle Groovy. Language: Java. Packaging: Jar. Java: 21.
- Spring Boot: a stable release offered by Initializr; avoid SNAPSHOT, milestone and release candidate builds.
- Group: com.example. Artifact and Name: backend. Package name: com.example.backend.
- Dependencies: Spring Web. Leave database and Spring Security dependencies out of this first example.

Generate and extract the ZIP so build.gradle and gradlew are directly inside hackathon-app/backend. Keep the generated build.gradle, settings.gradle, Gradle wrapper files and application class. Initializr provides compatible plugin and dependency versions; record them in Git instead of copying arbitrary version numbers from another tutorial. Java 21 is compatible with the current Spring Boot line. [8, 9]

Confirm build.gradle contains this Java toolchain setting:

Gradle Groovy

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
```

Create backend/src/main/resources/application.properties:

Properties

```
spring.application.name=backend
server.port=${PORT:8080}
```

Create backend/src/main/java/com/example/backend/HelloController.java:

Java 21

```
package com.example.backend;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    public record HelloResponse(String message) {}

    @GetMapping("/api/hello")
    public HelloResponse hello() {
        return new HelloResponse("Hello from Java 21");
    }
}
```

Keep this controller in the application package or a subpackage so Spring component scanning finds it. The record becomes a JSON response. Start the backend as shown in section 7, then visit <http://localhost:8080/api/hello>. Expect HTTP 200 and {"message":"Hello from Java 21"}.

4 Use a simple repository layout

Generated configuration files are omitted where they do not affect the overview. Create the optional folders only when a feature needs them.

Directory layout

```
hackathon-app/  
  README.md  
  .gitignore  
  .vscode/  
    extensions.json          optional team recommendations  
  frontend/  
    package.json  
    package-lock.json  
    vite.config.ts  
    index.html              HTML entry  
    tsconfig*.json  
    eslint.config.js  
    .env.example            public configuration template  
    public/                 assets copied without processing  
    src/  
      main.tsx              React entry and CSS import  
      App.tsx               top level UI  
      index.css             Tailwind import  
      components/           reusable UI when needed  
      pages/                screens when needed  
      lib/  
        api.ts              shared HTTP helper  
        assets/             imported images and icons  
  backend/  
    build.gradle  
    settings.gradle  
    gradlew  
    gradlew.bat  
    gradle/wrapper/         wrapper JAR and properties  
    src/main/java/com/example/backend/  
      BackendApplication.java  
      HelloController.java  
      config/WebConfig.java  optional CORS configuration  
      service/               business logic when needed  
      repository/            persistence when needed  
    src/main/resources/application.properties  
    src/test/java/com/example/backend/  
  docs/                     API notes and demo instructions
```

Keep controllers focused on HTTP input and output. Move business logic into services as it grows. Add repositories and a database only when persistence is part of the demo. Agree on endpoint paths and JSON fields before splitting frontend and backend work.

Add these entries to the root .gitignore, alongside the generated files:

Git ignore

```
*/node_modules/  
*/dist/  
*/.gradle/  
*/build/  
*/.env  
*/.env.*  
!*/.env.example
```

Commit source, package-lock.json and all Gradle wrapper files. Put prerequisites, launch commands, environment variables and the demo path in README.md. This repository uses two independent builds; no root package.json or npm workspace is required.

5 Connect React to the example endpoint

Create `frontend/src/lib/api.ts`. Leave `VITE_API_BASE_URL` unset locally so requests use the Vite proxy. The optional base URL supports a separately hosted backend later. Use an origin without a trailing slash.

TypeScript

```
const baseUrl = import.meta.env.VITE_API_BASE_URL ?? ''

export type HelloResponse = { message: string }

export async function getHello(): Promise<HelloResponse> {
  const response = await fetch(`${baseUrl}/api/hello`)
  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`)
  }
  return response.json()
}
```

Replace `frontend/src/App.tsx` with:

React TSX

```
import { useState } from 'react'
import { getHello } from '../lib/api'

export default function App() {
  const [message, setMessage] = useState('Ready to connect')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  async function connect() {
    setLoading(true)
    setError('')
    try {
      setMessage((await getHello()).message)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Request failed')
    } finally {
      setLoading(false)
    }
  }

  return (
    <main className="min-h-screen bg-slate-50 p-8 text-slate-900">
      <h1 className="text-3xl font-bold">Hackathon app</h1>
      <p className="my-4" aria-live="polite">{message}</p>
      <button onClick={connect} disabled={loading}
        className="rounded bg-blue-600 px-4 py-2 text-white disabled:opacity-50">
        {loading ? 'Connecting...' : 'Call Java API'}
      </button>
      {error && <p role="alert" className="mt-4 text-red-700">{error}</p>}
    </main>
  )
}
```

Run both services, open <http://localhost:5173> and click Call Java API. The page should display Hello from Java 21. In browser developer tools, the Network panel should show `/api/hello` returning JSON with status 200. TypeScript describes the expected shape; it does not validate incoming JSON at runtime.

6 Understand proxy and CORS choices

Use the relative `/api` URL and Vite proxy for local development. The browser talks only to `localhost:5173`, so this flow does not need backend CORS configuration. The server forwards the request to Java. Setting `Vite.server.cors` does not configure Spring's CORS policy. [7]

If the browser calls <http://localhost:8080> directly, or your deployed frontend and backend have different origins, configure CORS on the backend. An origin includes the scheme, host and port. `localhost` and `127.0.0.1` are different origins. [10]

For that alternative, create `backend/src/main/java/com/example/backend/config/WebConfig.java`:

Java 21

```
package com.example.backend.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Value("${app.frontend-origin:http://localhost:5173}")
    private String frontendOrigin;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins(frontendOrigin)
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
            .allowedHeaders("Content-Type", "Authorization");
    }
}
```

In production, set the backend environment variable `APP_FRONTEND_ORIGIN` to the exact frontend origin, such as <https://app.example.com>, without a path or trailing slash. This example does not use cookies or credentials. If you later add cookie authentication or Spring Security, configure credentials, CORS and CSRF together for that design.

Frontend environment configuration

Create `frontend/.env.example` with the following public setting. Teammates can copy it to `.env.local` if they need an override:

Properties

```
# Empty uses /api on the frontend origin
VITE_API_BASE_URL=
```

For separate hosting, set `VITE_API_BASE_URL=https://api.example.com` in the frontend build environment. Restart Vite after local environment changes; rebuild and redeploy after production changes. Vite embeds `VITE_` values in browser assets, so never store API secrets or database passwords there. Keep secrets in backend environment variables. [11]

7 Develop build and run

Open two VS Code terminals from hackathon-app. Use the commands for your operating system; do not run both backend variants.

Start development

Terminal A on Windows PowerShell:

PowerShell

```
cd backend
.\gradlew.bat bootRun
```

Terminal A on macOS or Linux:

Terminal

```
cd backend
chmod +x gradlew
./gradlew bootRun
```

Terminal B on either system:

Terminal

```
cd frontend
npm run dev
```

The first Gradle run downloads its distribution and dependencies. Wait for the backend startup message before testing the UI. Frontend edits update through Vite. Restart bootRun after Java edits unless you deliberately add and configure Spring Boot DevTools. Stop each service with Ctrl+C.

Validate and build

In frontend/, use npm install for the initial setup and intentional dependency changes. On a fresh clone or in CI with a committed lockfile, run:

Terminal

```
npm ci
npm run lint
npm run build
npm run preview
```

The React TypeScript template's build script performs TypeScript checking and creates frontend/dist. Preview normally serves at <http://localhost:4173>. It is a local build check, not a production server. For an API check through preview, add preview: { proxy: { '/api': 'http://localhost:8080' } } to the Vite config, or build with the separate API base URL and allow origin <http://localhost:4173> in backend CORS. [12]

In backend/ on Windows:

PowerShell

```
.\gradlew.bat clean build
java -jar build/libs/backend-0.0.1-SNAPSHOT.jar
```

On macOS or Linux, replace .\gradlew.bat with ./gradlew. The build task runs tests and creates an executable Boot JAR. Stop bootRun before launching it to free port 8080. Use the actual generated JAR filename if your project version differs; do not select a -plain.jar file. [9]

Before the demo, confirm the direct endpoint, browser request, loading and error states, frontend lint/build and backend build all work. The generated backend test is only a starting point; add endpoint and business behavior tests for real features.

8 Optional deployment and troubleshooting

Choose a hosting pattern

For a short hackathon, deploy the frontend's `dist/` folder to a static host and the Boot JAR to a Java 21 service or container host. Configure the frontend project root as `frontend`, install with `npm ci`, build with `npm run build`, and publish `dist`. Configure the backend project root as `backend`, build with `./gradlew clean build`, and start the executable Boot JAR with Java 21.

For separate origins, set `VITE_API_BASE_URL` before the frontend build and `APP_FRONTEND_ORIGIN` on the backend. Use HTTPS for both services. The `server.port=${PORT:8080}` setting accepts a hosting platform's assigned port. Ensure the process listens on the platform's network interface, not only `localhost`. The Vite dev proxy is not part of the deployed assets.

For one public origin, use a reverse proxy that serves the frontend and forwards `/api` to Java, or copy the built frontend into `backend/src/main/resources/static` before building the JAR. Leave `VITE_API_BASE_URL` empty. If you add client side routes, configure an `index.html` fallback for those routes while preserving API errors and real asset paths.

Keep credentials in the host's backend secret settings. Check the public `/api/hello` endpoint and the deployed browser request after release. Hosting providers differ in build commands, runtime versions and network settings; use the same committed lockfile and Gradle wrapper as development.

Fix common setup problems

- Tool not found: restart VS Code, verify `PATH` and `JAVA_HOME`, and check that `javac` is installed as well as `java`.
- Gradle Java mismatch: run `./gradlew.bat --version` or `./gradlew --version`. Verify JDK 21 and the generated wrapper. Java 21 needs Gradle 8.5+ to run Gradle; respect any higher minimum required by your Boot version. [13]
- Port in use: stop the other process. Vite `strictPort` intentionally fails rather than silently moving away from 5173. If the backend port changes, update the proxy target.
- API 404: check `/api/hello`, the controller package and the running backend. Open the backend URL directly to isolate routing from the frontend.
- Proxy connection refused: start Java and confirm it is listening on 8080. Restart Vite after editing `vite.config.ts`.
- CORS error: verify the actual browser origin and backend policy. Use the local proxy or explicitly allow the separate origin. Do not disable browser protections.
- No Tailwind styles: verify the Vite plugin, the CSS import and `main.tsx`'s stylesheet import. Remove leftover `App.css` imports and use full class names.
- Dependency conflict: check the installed Vite and Tailwind plugin peer requirements. Choose compatible releases and commit the lockfile; avoid forcing an incompatible install.
- Blank production route or API failure: check the static host fallback, Vite base path if hosted under a subpath, build time API URL, HTTPS and deployed CORS settings.

9 Official references

Setup commands and compatibility guidance checked on 9 October 2026. Use the project lockfile and generated Gradle files to preserve the versions your team actually installs. The numbered references support the related sections; code examples and project organization are adapted for this guide.

- [1] Visual Studio Code setup and downloads
<https://code.visualstudio.com/docs/setup/setup-overview>
- [2] Node.js downloads and Git downloads
<https://nodejs.org/en/download>
<https://git-scm.com/downloads/>
- [3] Vite getting started and supported Node.js versions
<https://vite.dev/guide/>
- [4] Eclipse Temurin JDK installation
<https://adoptium.net/installation/>
- [5] Spring Boot and Java tooling in VS Code
<https://code.visualstudio.com/docs/java/java-spring-boot>
- [6] Tailwind CSS installation with Vite
<https://tailwindcss.com/docs/installation/using-vite>
- [7] Vite server options and development proxy
<https://vite.dev/config/server-options>
- [8] Spring Initializr and Spring Boot system requirements
<https://start.spring.io>
<https://docs.spring.io/spring-boot/system-requirements.html>
- [9] Building a Spring Boot application
<https://spring.io/guides/gs/spring-boot/>
- [10] Spring CORS guide
<https://spring.io/guides/gs/rest-service-cors/>
- [11] Vite environment variables and modes
<https://vite.dev/guide/env-and-mode>
- [12] Vite production builds and static deployment
<https://vite.dev/guide/static-deploy.html>
- [13] Gradle Java compatibility matrix
<https://docs.gradle.org/current/userguide/compatibility.html>